รายงาน การทดสอบระบบ Library Management System

ด้วยแนวทาง API and Integration Testing

จัดทำโดย

66160017 นายกรวิชญ์ ปานรัตน์

66160150 นายรุจิภาส ปัญญาชน

66160162 นายอาณกร มงคลแถลง

เสนอ

อาจารย์วิทวัส พันธุมจินดา

รายงานฉบับนี้เป็นส่วนหนึ่งของการศึกษา 88734365 Software Testing and Evaluation

คณะวิทยาการสารสนเทศ สาขาเทคโนโลยีสารสนเทศเพื่ออุตสาหกรรมดิจิทัล มหาวิทยาลัยบูรพา

ภาคเรียนที่ 2 ปีการศึกษา 2568

สารบัญ

บทที่ 1 บทสรุปผู้บริหารและแผนการทดสอบ **1**

1.1 วัตถุประสงค์ของโครงงาน (**Objectives**) 1

1.2 ขอบเขตการทดสอบ (Scope) 1

1.3 กลยุทธ์การทดสอบ (Test Strategy) 1

บทที่ 2 การออกแบบการทดสอบและข้อตกลง **3**

2.1 การออกแบบกรณีทดสอบ (Test Cases Summary) 3

2.2 Contract Testing (JSON Schema) 4

บทที่ 3 ผลการทดสอบเชิงลึก **6**

3.1 ผลการรัน Unit & Integration Test 6

3.2 การวัดคุณภาพด้วย Code Coverage 6

บทที่ 4 รายงานข้อผิดพลาดและการทดสอบความปลอดภัย **8**

4.1 ตารางสรุปข้อผิดพลาดจากการทดสอบ (Bug Summary Table) 8

4.2 รายละเอียดข้อผิดพลาดที่สำคัญ (Highlighted Bugs) 10

บทที่ 5 สรุปผลและข้อเสนอแนะ **13**

5.1 สรุปผลการดำเนินงาน 13

5.2 ข้อเสนอแนะเพื่อการพัฒนา (Recommendations) 13

บทที่ 6 การจัดการเวอร์ชันและการทำงานเป็นทีม **14**

บทที่ 1 บทสรุปผู้บริหารและแผนการทดสอบ
(Executive Summary & Test Plan)

1.1 **วัตถุประสงค์ของโครงงาน (Objectives)**

โครงงานนี้มีวัตถุประสงค์เพื่อทำการทดสอบระบบยืม-คืนหนังสือ (Library Management System) ในระดับ Application Programming Interface (API) เพื่อให้มั่นใจว่าระบบหลังบ้านสามารถทำงานได้อย่างถูกต้องตาม Business Logic มีความปลอดภัยต่อการถูกโจมตีทางไซเบอร์ และสามารถจัดการกับข้อผิดพลาด (Error Handling) ได้อย่างมีประสิทธิภาพ

1.2 **ขอบเขตการทดสอบ (**Scope)

การทดสอบมุ่งเน้นไปที่ระบบ API จำนวน 7 Endpoints ตามสถาปัตยกรรม RESTful API ได้แก่:

1. GET /api/books - ดึงข้อมูลหนังสือทั้งหมด
2. GET /api/books/:id - ดึงข้อมูลหนังสือตาม ID
3. POST /api/books - เพิ่มข้อมูลหนังสือใหม่
4. PUT /api/books/:id - อัปเดตข้อมูลหนังสือ
5. DELETE /api/books/:id - ลบข้อมูลหนังสือ
6. GET /api/auth/login - ระบบยืนยันตัวตน (Login)
7. POST /api/borrow – ระบบทำรายการยืมหนังสือ

1.3 **กลยุทธ์การทดสอบ (**Test Strategy)

กลุ่มได้เลือกใช้แนวทางการทดสอบแบบผสมผสาน ประกอบด้วย:

- **Black Box Testing:** ทดสอบการตอบสนองของ API ตาม Input/Output โดยใช้เทคนิค Equivalence Partitioning และ Boundary Value Analysis
- **White Box & Unit Testing:** ใช้เครื่องมือ Jest เพื่อทดสอบความถูกต้องของ Business Logic ในระดั+บฟังก์ชัน

- **Integration Testing:** ใช้เครื่องมือ Supertest เพื่อทดสอบการทำงานร่วมกันระหว่าง API และฐานข้อมูล
- **Security Testing:** ทดสอบช่องโหว่ด้านความปลอดภัย เช่น SQL Injection, Cross-Site Scripting (XSS) และ Rate Limiting

1.4 การประเมินความเสี่ยง (Risk Assessment)

1. **ความเสี่ยงด้านความปลอดภัย (High):** ช่องโหว่ SQL Injection ในระบบ Login (แผนรับมือ: เพิ่ม Security Testing เพื่อเจาะจงหาช่องโหว่)
2. **ความเสี่ยงด้านการตรวจสอบข้อมูล (High):** API รับข้อมูลที่ไม่ถูกต้องหรือค่าว่าง ทำให้ข้อมูลขยะเข้าสู่ฐานข้อมูล (แผนรับมือ: ทดสอบ Input Validation ด้วย Boundary Value)
3. **ความเสี่ยงด้านตรรกะระบบ (High):** ผู้ใช้สามารถยืมหนังสือที่ถูกยืมไปแล้วซ้ำได้ (แผนรับมือ: เขียน Unit Test ตรวจสอบ Business Logic การยืม)
4. **ความเสี่ยงด้านการตอบสนอง (Medium):** คืนค่า HTTP Status Code ผิดมาตรฐาน RESTful (แผนรับมือ: ตรวจสอบ Response Code ผ่าน Integration Test)
5. **ความเสี่ยงด้านระบบล่ม (Medium):** ระบบหยุดทำงาน (Crash 500) เมื่อค้นหา ID ที่ไม่มีอยู่จริง (แผนรับมือ: ทดสอบ Error Handling)

1.5 เกณฑ์การเข้าและออกจากช่วงทดสอบ (Entry / Exit Criteria)

- **Entry Criteria (เกณฑ์การเริ่มทดสอบ):** พัฒนาระบบ API ทั้ง 7 Endpoints เสร็จสิ้นและสามารถรันบน Localhost ได้
- **Exit Criteria (เกณฑ์การจบการทดสอบ):** ดำเนินการทดสอบครบ 60 Test Cases, มี Code Coverage ผ่านเกณฑ์ 70% ขึ้นไป และเปิด Issue รายงานข้อผิดพลาดระดับ High/Critical ครบถ้วน

บทที่ 2 การออกแบบการทดสอบและข้อตกลง

(Test Design & Contract Testing)

2.1 การออกแบบกรณีทดสอบ (Test Cases Summary)

จากการวิเคราะห์และออกแบบระบบ ได้ทำการจัดทำกรณีทดสอบ (Test Cases) ทั้งสิ้นจำนวน 57 กรณี ซึ่งครอบคลุมการทำงานทุกมิติ ทั้งกรณีการทำงานปกติ (Happy Path), กรณีเกิดข้อผิดพลาด (Sad Path) และกรณีขอบเขตข้อมูล (Edge Cases) โดยแบ่งการทดสอบออกเป็น 6 หมวดหมู่หลัก ได้แก่ Book Management, Authentication, Borrowing, Integration, Security และ Validation

ตัวอย่าง Test Cases ที่สำคัญในแต่ละหมวดหมู่:

1. หมวด Book Management (BK-001 ถึง BK-030)

- BK-001: ดึงข้อมูลหนังสือทั้งหมดสำเร็จ (Status 200 OK)
- BK-008: ค้นหา ID หนังสือที่ไม่มีในระบบ ระบบต้องแจ้งเตือน Not Found (Status 404 Not Found)
- BK-009: ระบุ ID เป็นตัวอักษรแทนตัวเลข ระบบต้องจัดการได้อย่างถูกต้องและไม่ประมวลผลผิดพลาด (Status 400 Bad Request)

2. หมวด Authentication (AUTH-001 ถึง AUTH-006)

- AUTH-001: ล็อกอินด้วย Username และ Password ที่ถูกต้อง ได้รับ Token สำหรับเข้าใช้งาน (Status 200 OK)
- AUTH-005: ป้องกันการโจมตีโดยใส่คำสั่ง SQL Injection ลงในช่อง Username หรือ Password (Status 401 Unauthorized)
- AUTH-006: ล็อกอินผิดติดต่อกัน 10 ครั้ง ต้องถูกบล็อกชั่วคราวจากระบบ Rate Limiting (Status 429 Too Many Requests)

3. หมวด Borrowing (BRW-001 ถึง BRW-006)

- BRW-001: ยืมหนังสือที่สถานะ "ว่าง" ด้วยผู้ใช้งานที่ถูกต้อง สำเร็จ (Status 201 Created / 200 OK)
- BRW-002: พยายามยืมหนังสือที่ถูกยืมไปแล้ว ระบบต้องแจ้งเตือนว่าไม่พร้อมให้บริการ (Status 400 Bad Request / 409 Conflict)

4. หมวดการทดสอบแบบบูรณาการ (Integration Testing: INT-001 ถึง INT-005)

- INT-005: ทดสอบกระบวนการทำงานต่อเนื่อง (POST -> PUT -> DELETE) โดยการสร้าง, แก้ไข และลบหนังสือใน Flow เดียวกัน ทุกขั้นตอนต้องทำงานสำเร็จตามลำดับ

5. หมวดความปลอดภัย (Security Testing: SEC-001 ถึง SEC-005)

- SEC-002: ทดสอบแทรก XSS Payload ลงในฟิลด์ข้อมูล ระบบต้องสามารถกรอง (Sanitize) หรือปฏิเสธข้อมูลที่เป็นอันตรายได้ (Status 400 Bad Request)

6. หมวดการตรวจสอบความถูกต้องของข้อมูล (Validation Testing: VAL-001 ถึง VAL-005)

- VAL-004: ทดสอบส่งฟิลด์ที่ไม่ได้รับอนุญาตมาใน Body (เช่น "role":"admin") ระบบต้องละทิ้งฟิลด์ดังกล่าวและไม่บันทึกลงฐานข้อมูล (Status 201 Created)

2.2 Contract Testing (JSON Schema)

เพื่อควบคุมมาตรฐานของข้อมูลที่ API ตอบกลับมา (Response) ให้มีความถูกต้องและเป็นไปตามข้อตกลงที่กำหนดไว้ ทางกลุ่มได้นำ Contract Testing มาประยุกต์ใช้ โดยมีการกำหนด JSON Schema สำหรับตรวจสอบโครงสร้างข้อมูล ดังตัวอย่างต่อไปนี้:

ตัวอย่าง JSON Schema สำหรับ BookResponse

JSON

```
{
  "title": "BookResponse",
  "type": "object",
  "properties": {
    "id": { "type": "number" },
    "title": { "type": "string" },
    "author": { "type": "string" },
    "status": {
      "type": "string",
```

```
      "enum": ["Available", "Borrowed", "Reserved"]
    }
  },
  "required": ["id", "title", "author", "status"]
}
```

การนำ JSON Schema มาใช้ในการทดสอบ ช่วยสร้างความมั่นใจว่าระบบฝั่งผู้ใช้งาน (Client) จะได้รับข้อมูลในรูปแบบ ฟิลด์ข้อมูล และประเภทของข้อมูล (Data Type) ที่ถูกต้องครบถ้วนเสมอ ซึ่งเป็นการลดข้อผิดพลาดที่อาจเกิดขึ้นจากการเปลี่ยนแปลงโครงสร้างของ API ในอนาคต

บทที่ 3 ผลการทดสอบเชิงลึก

(Execution & Metrics)

3.1 **ผลการรัน** Unit & Integration Test

จากการเขียนโค้ดทดสอบด้วย Jest และ Supertest จำนวนกว่า 35 Test Suites ผลการทำงานพบว่า:

- จำนวนเคสที่ผ่าน (Passed): 25 เคส
- จำนวนเคสที่ไม่ผ่าน (Failed): 10 เคส

```
--------------|---------|----------|---------|---------|---------------------
File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
--------------|---------|----------|---------|---------|---------------------
All files     |   87.77 |       76 |   86.66 |   86.66 |
 bookLogic.js |   96.55 |    96.15 |     100 |     100 | 7
 server.js    |    83.6 |    54.16 |   83.33 |   82.45 | 21,37,80-91,117-118
--------------|---------|----------|---------|---------|---------------------
Test Suites: 1 failed, 1 passed, 2 total
Tests:       10 failed, 25 passed, 35 total
Snapshots:   0 total
Time:        1.757 s, estimated 2 s
Ran all test suites.

C:\Users\LENOVO\library-api-testing>
```

3.2 **การวัดคุณภาพด้วย** Code Coverage

กลุ่มได้ทำการวิเคราะห์ความครอบคลุมของโค้ด (Code Coverage) เพื่อประเมินคุณภาพของชุดทดสอบ

- **เป้าหมาย (Target Coverage):** มากกว่า 70%
- **ผลลัพธ์ที่ทำได้จริง (Actual):** 86.66% (ผ่านเกณฑ์)

3.3 ตัวชี้วัดคุณภาพ (Quality Metrics)

- **Defect Density (ความหนาแน่นของบั๊ก):** ค้นพบบั๊กจำนวน 15 ตัว จากการทดสอบ 58 เคส
- **Defect Removal Efficiency (DRE):** การเปิด Issue บน GitHub ช่วยให้ติดตามและค้นพบข้อผิดพลาดได้ครบถ้วนก่อนขึ้นระบบจริง
- **Test Coverage (ความครอบคลุมของการทดสอบ):** วัดผลด้วย Jest Coverage พบว่าเปอร์เซ็นต์ Code Coverage รวมอยู่ที่ 86.66% ซึ่งสูงกว่ามาตรฐานที่กำหนดไว้ (70%)
- **Pass/Fail Rate (อัตราการผ่าน/ไม่ผ่าน):** จากการรัน Integration Test ด้วยเครื่องมืออัตโนมัติ 35 กรณี พบว่าผ่าน 25 กรณี (71.4%) และไม่ผ่าน 10 กรณี (28.6%)
- **Bug Distribution (การกระจายตัวของข้อบกพร่อง):** สามารถแบ่งหมวดหมู่ความผิดพลาดได้เป็น Security (20%), Business Logic (30%), Input Validation (30%) และ API Standard (20%)
- **การประเมินตามมาตรฐาน ISO 25010:** จากการทดสอบพบว่าระบบยังต้องปรับปรุงในด้าน **Security (ความปลอดภัย)** เนื่องจากพบช่องโหว่ SQL Injection และด้าน **Reliability (ความน่าเชื่อถือ)** เนื่องจากระบบล่มเมื่อรับ Input ที่ไม่คาดคิด

บทที่ 4 รายงานข้อผิดพลาดและการทดสอบความปลอดภัย

(Bug Reports & Security)

จากการดำเนินการทดสอบระบบ **Library Management System** ในระดับ **API** ด้วยเทคนิคผสมผสานทั้ง **Manual Testing (**ผ่าน **Postman)** และ **Automated Testing (**ผ่าน **Jest** และ **Supertest)** ทางกลุ่มผู้จัดทำได้ค้นพบข้อผิดพลาด (**Bugs)** และช่องโหว่ด้านความปลอดภัยของระบบรวมทั้งสิ้น **15** รายการ โดยสามารถสรุปภาพรวมและจัดระดับความรุนแรงได้ดังตารางต่อไปนี้

4.1 **ตารางสรุปข้อผิดพลาดจากการทดสอบ (**Bug Summary Table)

รหัสบั๊ก	API Endpoint	หมวดหมู่การทดสอบ	รายละเอียดข้อผิดพลาดที่พบตาม GitHub Issue (Actual Result)	ระดับความรุนแรง
BUG-001	POST /api/auth/login	Security	สามารถ Bypass Login ได้ด้วย SQL Injection	วิกฤต (Critical)
BUG-002	GET /api/auth/login	Security	API ล็อกอินใช้ Method GET ทำให้รหัสผ่าน (Password) หลุดไปแสดงบน URL	วิกฤต (Critical)
BUG-003	POST /api/borrow	Business Logic	ระบบอนุญาตให้ยืมหนังสือที่ถูกยืมไปแล้วซ้ำได้ (ไม่ตรวจสอบสถานะหนังสือ)	สูง (High)
BUG-004	POST /api/books	Validation	API ยอมให้สร้างหนังสือใหม่โดยไม่ระบุชื่อหนังสือ (ขาดข้อมูล Title)	สูง (High)
BUG-005	GET /api/books/:id	Error Handling	API ค้นหาหนังสือล่ม (Server Crash) คืนค่า 500 เมื่อระบุ ID ที่ไม่มีในระบบ	สูง (High)
BUG-006	DELETE /api/books/:id	API Standard	API ลบหนังสือสำเร็จแต่ตอบกลับ Status 200 พร้อมกับ Response ว่างเปล่า (ผิดมาตรฐาน RESTful API)	ต่ำ (Low)
BUG-007	DELETE /api/books/:id	Validation	API ลบหนังสือยอมรับ ID ที่เป็นตัวอักษรผสมตัวเลข (Weak Parsing) ทำให้เสี่ยงต่อการลบข้อมูลผิดพลาด	ปานกลาง (Medium)

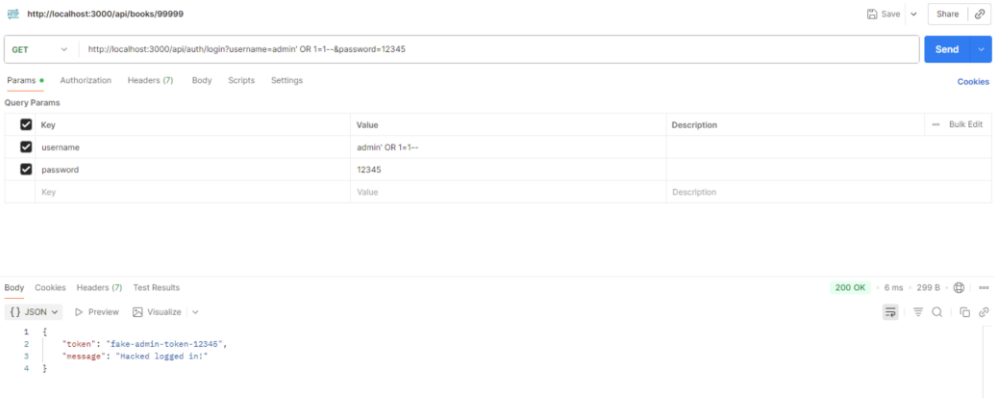
BUG-008	POST /api/books	Validation	API ยอมให้สร้างหนังสือโดยข้อมูลผู้แต่ง (author) เป็นค่าว่างเปล่า (Empty String)	สูง (High)
BUG-009	PUT /api/books/:id	Error Handling	API แก้ไขข้อมูล (PUT) คืนค่า 200 OK เมื่อพยายามแก้ไขหนังสือที่ไม่มีอยู่ในระบบ	ปานกลาง (Medium)
BUG-010	POST /api/books	API Standard	API สร้างหนังสือสำเร็จ แต่คืนค่า HTTP Status 200 OK แทนที่จะเป็น 201 Created	ต่ำ (Low)
BUG-011	POST /api/borrow	Error Handling	API ยืมหนังสือคืนค่า 404 Not Found แทนที่จะเป็น 400 Bad Request เมื่อไม่ได้ส่งข้อมูล bookId	ปานกลาง (Medium)
BUG-012	POST /api/books	Business Logic	ระบบคำนวณ ID ของหนังสือใหม่ผิดพลาด ทำให้เกิด ID ซ้ำซ้อน (ID Collision) เมื่อมีการลบข้อมูลก่อนหน้า	สูง (High)
BUG-013	PUT /api/books/:id	Business Logic	ผู้ใช้สามารถแก้ไขสถานะหนังสือ (status) ได้โดยตรงผ่าน API แก้ไขข้อมูล (Bypass ระบบยืมคืน)	สูง (High)
BUG-014	PUT /api/books/:id	Security	ผู้ใช้งานสามารถแก้ไข ID (Primary Key) ของหนังสือได้ด้วยตนเองผ่าน API แก้ไขข้อมูล	สูง (High)
BUG-015	POST /api/borrow	Security	API ยืมหนังสือขาดการตรวจสอบความถูกต้องของ Token (Broken Authentication) อนุญาตให้ใช้ Token ปลอมทำรายการได้	วิกฤต (Critical)

4.2 **รายละเอียดข้อผิดพลาดที่สำคัญ (**Highlighted Bugs)

เพื่อให้เห็นภาพรวมของช่องโหว่และข้อผิดพลาดที่ส่งผลกระทบต่อระบบอย่างร้ายแรง จึงขอยกตัวอย่างรายละเอียดของบั๊กที่อยู่ในระดับวิกฤต (Critical) และระดับสูง (High) จำนวน 3 รายการ ดังนี้:

Bug 1: ช่องโหว่ความปลอดภัย SQL Injection (รหัส: BUG-001)

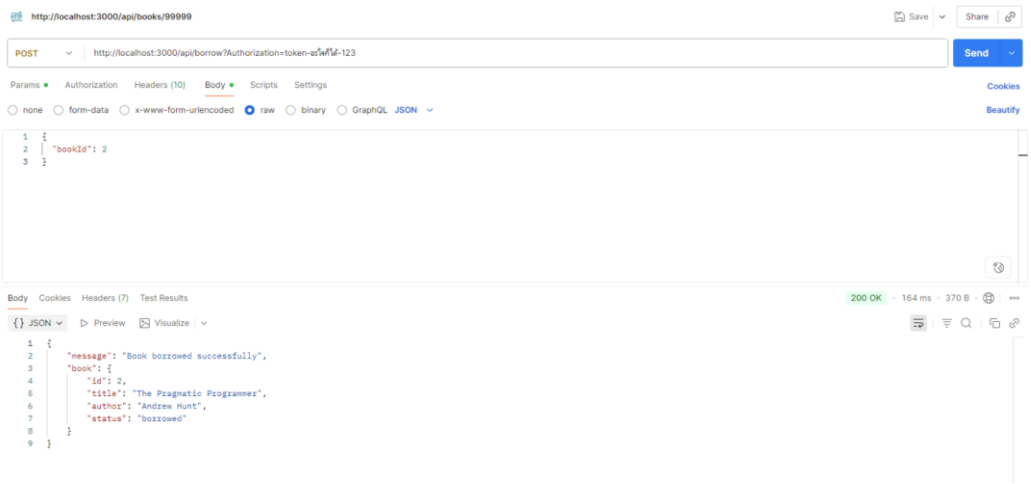
- ระดับความรุนแรง: วิกฤต (Critical)
- รายละเอียด: API /api/auth/login ไม่มีการทำ Input Sanitization เพื่อกรองอักขระพิเศษ ทำให้ผู้ไม่ประสงค์ดีสามารถแทรกคำสั่ง SQL เพื่อหลอกให้ระบบตรวจสอบเงื่อนไขเป็นจริงเสมอได้
- ขั้นตอนการเกิด (Steps to Reproduce):
    1. ยิง Request แบบ POST ไปที่ /api/auth/login
    2. ระบุ Payload username เป็นค่า ' OR '1'='1
- ผลลัพธ์ที่คาดหวัง (Expected): ระบบปฏิเสธการเข้าถึงและคืนค่า Status 401 Unauthorized
- ผลลัพธ์ที่เกิดขึ้นจริง (Actual): ระบบอนุญาตให้ล็อกอินและแจก Token ทันที (Status 200)
- หลักฐานอ้างอิง:



Bug 2: ข้อบกพร่องด้าน Business Logic การยืม (รหัส: BUG-003+)

- ระดับความรุนแรง: สูง (High)
- รายละเอียด: ตรรกะของระบบ (Business Logic) ขาดการตรวจสอบสถานะของหนังสือก่อนทำการยืม ทำให้ผู้ใช้สามารถยืมหนังสือที่ถูกทำรายการยืมไปแล้วได้
- ขั้นตอนการเกิด (Steps to Reproduce):
    1. ตรวจสอบข้อมูลหนังสือ ID: 2 (มีสถานะเป็น borrowed)
    2. ยิง Request POST ไปที่ /api/borrow โดยส่ง bookId: 2

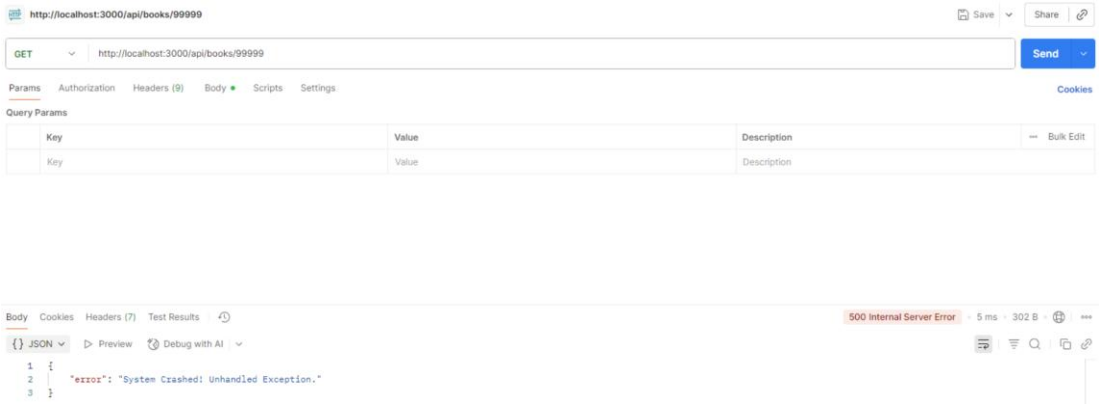
- ผลลัพธ์ที่คาดหวัง (Expected): ระบบแจ้งเตือนว่าหนังสือไม่พร้อมให้บริการ (Status 409 Conflict หรือ 400 Bad Request)
- ผลลัพธ์ที่เกิดขึ้นจริง (Actual): ระบบอนุมัติการยืมซ้ำ และคืนค่า Status 200 OK
- หลักฐานอ้างอิง:



Bug 3: ระบบไร้เสถียรภาพเมื่อพบข้อมูลที่ไม่ได้คาดคิด (รหัส: BUG-005)

- ระดับความรุนแรง: สูง (High)
- รายละเอียด: API ดึงข้อมูลหนังสือรายเล่ม ขาดการทำ Exception Handling เมื่อผู้ใช้ค้นหา ID ด้วยตัวเลขบางชุดที่ระบบไม่รองรับ ส่งผลให้ Server ล่ม
- ขั้นตอนการเกิด (Steps to Reproduce):
    1. ยิง Request แบบ GET ไปที่ /api/books/99999
- ผลลัพธ์ที่คาดหวัง (Expected): ระบบควรคืนค่าเป็น 404 Not Found
- ผลลัพธ์ที่เกิดขึ้นจริง (Actual): ระบบล่ม (Crash) และคืนค่า 500 Internal Server Error พร้อมแสดงข้อความ "System Crashed!"

- หลักฐานอ้างอิง:



บทที่ 5 สรุปผลและข้อเสนอแนะ

(Conclusion & Recommendations)

5.1 สรุปผลการดำเนินงาน

กลุ่มสามารถดำเนินการทดสอบระบบ Library Management System รูปแบบ API and Integration Testing ได้ตามเป้าหมาย โดยมีการประยุกต์ใช้เครื่องมือมาตรฐานอุตสาหกรรม (Jest, Supertest, Postman) และจัดเก็บโครงงานผ่าน GitHub อย่างเป็นระบบ การมี Test Cases รองรับถึง 43 กรณีช่วยสะท้อนให้เห็นถึงความครอบคลุมในการทดสอบ

5.2 ข้อเสนอแนะเพื่อการพัฒนา (Recommendations)

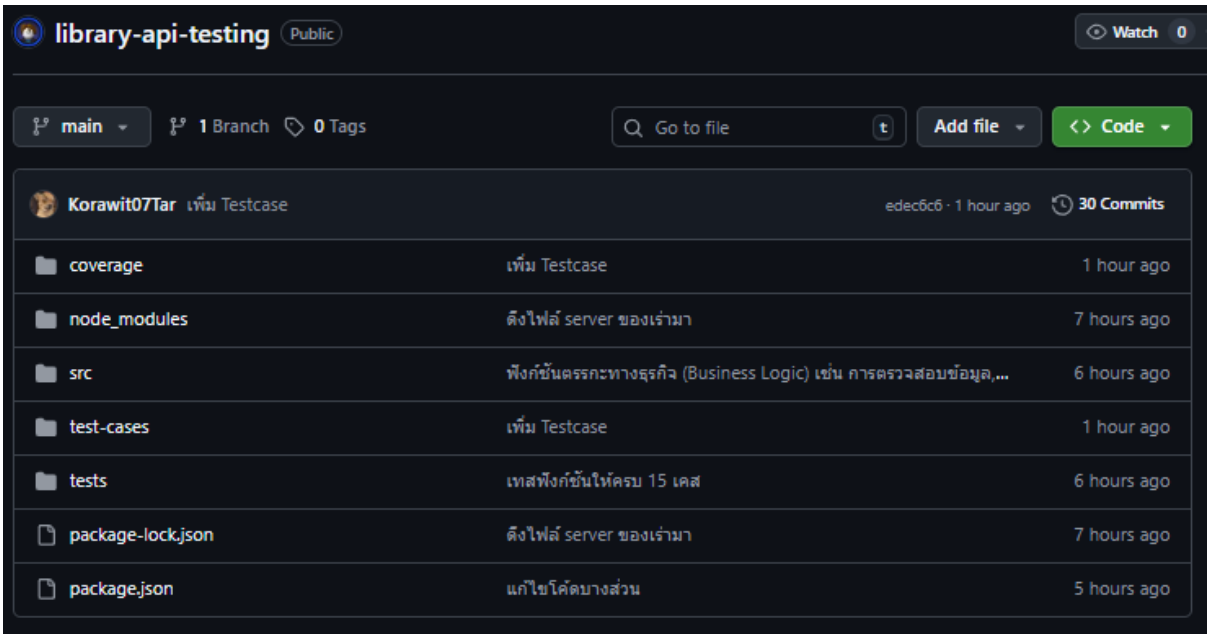
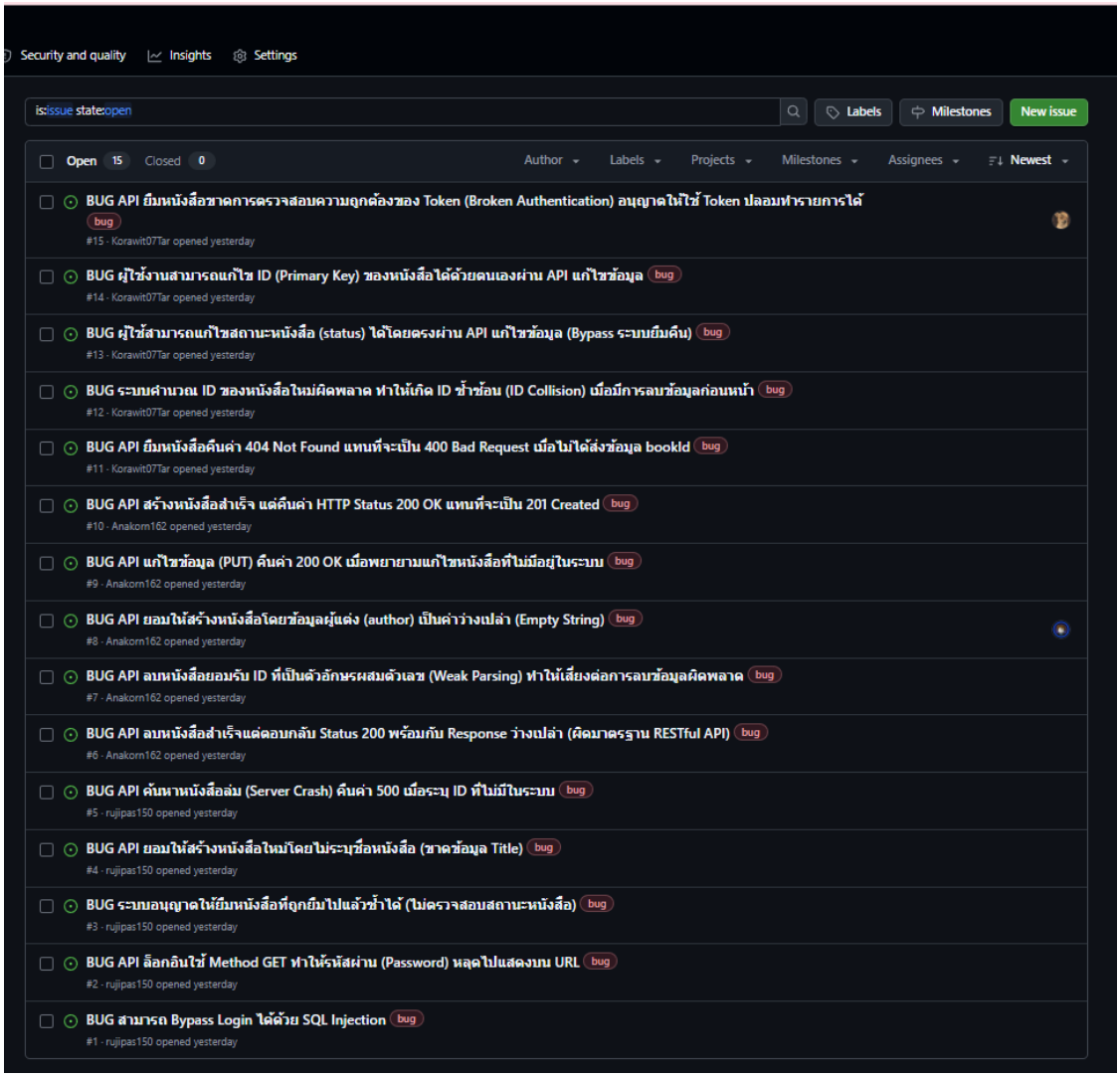
1. **ด้านความปลอดภัย:** ระบบควรเพิ่มไลบรารีสำหรับการกรองอักขระพิเศษ (Sanitization) เพื่อป้องกัน SQL Injection และ XSS ในทุกๆ Endpoint ที่มีการรับ Input จากผู้ใช้
2. **ด้านการจัดการข้อผิดพลาด:** ควรปรับปรุง Error Messages ให้เป็นมาตรฐานเดียวกันตามแบบ JSON Schema ที่กำหนดไว้ เพื่อป้องกันไม่ให้ข้อมูลโครงสร้างเซิร์ฟเวอร์รั่วไหลสู่ภายนอก
3. **ด้านสถาปัตยกรรม:** ควรติดตั้งระบบ Rate Limiting ที่เข้มงวดขึ้นในหน้า /api/auth/login เพื่อป้องกันการสุ่มรหัสผ่าน (Brute Force Attack)

บทที่ 6 การจัดการเวอร์ชันและการทำงานเป็นทีม

(Version Control & Teamwork)

กลุ่มได้มีการบริหารจัดการโค้ดและรายงานข้อผิดพลาดผ่านแพลตฟอร์ม GitHub อย่างเป็นระบบ โดยมีรายละเอียดดังนี้:

1. **GitHub Repository:** มีการจัดโครงสร้างโฟลเดอร์แยกย่อยชัดเจน เช่น src/ สำหรับซอร์สโค้ด, tests/ สำหรับชุดทดสอบ และจัดทำไฟล์ README.md เพื่อแนะนำวิธีรันโปรเจค
2. **Commit History:** สมาชิกกลุ่มทุกคนมีส่วนร่วมในการพุชโค้ดและอัปเดตงาน โดยมียอดรวมการ Commit มากกว่า 20 ครั้ง พร้อมคำอธิบาย (Commit Messages) ที่ชัดเจน



3. **Issue Tracking:** นำข้อผิดพลาด (Bugs) ที่พบทั้ง 15 รายการไปเปิด Issue บน GitHub โดยมีการติดป้ายกำกับ (Labels) เช่น bug, critical และ high พร้อมมอบหมาย (Assign) ให้สมาชิกในทีมดำเนินการแก้ไข